CS367 — Software Analysis

Competency Exam

Bryn Mawr College · Fall 2026

Setup

You are given a starter project, `cs367-competency/`. Unpack it.

Honor code. Complete this exam on your own. No outside sources, collaborators, or AI assistants are permitted. Please abide by the honor code.

Java version

Confirm your tools. You need a JDK 11 or newer:

```
$ javac -version     # need 11 or newer
$ java -version
```

On **goldengate** the default Java is already 11, so you are set. On your own machine, if `javac -version` reports something older than 11, use SDKMAN (`https://sdkman.io`) to get a newer one:

```
$ curl -s "https://get.sdkman.io" | bash
$ source "$HOME/.sdkman/bin/sdkman-init.sh"
$ sdk install java 11.0.21-open   # sdk list java  shows all versions
$ sdk use java 11.0.21-open       # this shell; 'sdk default' to persist
```

Project layout

```
src/Stats.java        class under test (median is unimplemented)
test/StatsTest.java   tests
lib/tinytest.jar      the external dependency (assertion library)
AnalyzerRun.java      two static checkers over labeled methods (Part 2)
```

Reference: the classpath

Reminder: `-cp` (or `-classpath`) is how you tell `javac` and `java` where to find classes outside your own source. Entries are separated by `:` (macOS/Linux) or `;` (Windows).

Part 1 — Compiling and running tests

1a. Compile against the dependency

Compile `src/Stats.java` and `test/StatsTest.java`.

Deliverable: the `.class` files.

1b. Run the tests and read the failure

Run the compiled `StatsTest`. You should see two passing tests (`mean`, `max`) and two failing (`median`).

1c. Fix the code, recompile, retest

Implement `Stats.median` in `src/Stats.java`. Median of a sorted list is the middle element (odd length) or the average of the two middle elements (even length). Recompile and rerun until all four tests pass.

Deliverable: submit `src/Stats.java` on Gradescope.

Part 2 — Evaluating analyzers

To judge a program analysis tool you compare its verdicts against ground truth. `AnalyzerRun.java` runs two static null-pointer checkers, **A** and **B**, over the same labeled methods. For each method it prints the truth (is it *actually* buggy) and each tool's verdict (`WARN` = the tool thinks the method can dereference null, `OK` = the tool thinks it is safe):

```
$ javac AnalyzerRun.java
$ java AnalyzerRun
```

Treat a `WARN` as the tool raising a *positive*. Submit your answers in a plain-text file named `part2.txt`.

2a. Count the outcomes

For *each* tool, classify every method into one of four categories:

- **True positive (TP):** `WARN`, and the method is buggy.
- **False positive (FP):** `WARN`, but the method is fine.
- **True negative (TN):** `OK`, and the method is fine.
- **False negative (FN):** `OK`, but the method is buggy.

Report the four counts for tool A and for tool B.

2b. Which tool is better?

1. For each tool: of all the warnings it raised, how many were false alarms? How many real bugs did it miss entirely?
2. In a few sentences, explain the trade-off between the two tools using your counts.
3. Name a situation where you would prefer tool A, and one where you would prefer tool B. Justify each with the counts.

Part 3 — Logical readiness

Written work. Submit your answers in a plain-text file named `part3.txt`. Show your reasoning.

3.1 Evaluating a predicate

Consider the predicate

$$P(x, y) \;=\; (x > 0) \wedge (x + y \leq 10) \;\vee\; (y = 0).$$

Evaluate P on each of the following inputs and give the truth value:

1. $(x, y) = (3, 4)$
2. $(x, y) = (3, 9)$
3. $(x, y) = (-1, 0)$
4. $(x, y) = (6, 5)$

3.2 Implication between formulae

For each pair, state whether the first formula implies the second (i.e. whether every assignment making the first true also makes the second true). If it does not, give a counterexample assignment.

1. Does $(p \to q) \wedge (q \to r)$ imply $p \to r$?
2. Does $(p \vee q) \wedge \neg p$ imply q?
3. Does $p \to q$ imply $q \to p$?

3.3 Truth tables and equivalence

1. Build the full truth table for $(p \wedge q) \to r$.
2. Two formulae are *logically equivalent* iff they have the same truth table. Show that $p \to q$ is equivalent to $\neg p \vee q$.
3. Use a truth table to show that $\neg(a \wedge b)$ is equivalent to $\neg a \vee \neg b$.

What to submit

Submit the following on Gradescope:

- **Part 1:** `Stats.class` and `StatsTest.class` from 1a, and `src/Stats.java` with your completed `median`.
- **Part 2:** `part2.txt` with your answers to 2a and 2b.
- **Part 3:** `part3.txt` with your answers to 3.1–3.3.

End of exam.